SIG Principal C# / WPF — Expanded Study Guide

Audience: Engineers preparing for a Principal-level interview who may be **rusty on CLR, GC, WPF, or systems design**. Every section explains concepts from first principles before interview depth.

Company context: SIG (Susquehanna International Group) and similar trading firms build **desktop C# / WPF tools** that show live market data, orders, and risk. Performance, correctness, and clear architectural reasoning matter as much as syntax.

How to study: Read each section's *Plain English* block first, then *Technical depth*, then *SIG / trading angle*, then *Interview phrases*.

Table of Contents

Part I — CLR & Type System

1. Object Model & Memory Layout

Plain English — start here if you're rusty

When you write `var order = new Order()`, the computer does two things:

1. It puts a **small pointer** (reference) in memory tied to your current method — often on the **stack** (think: scratch pad for the current function call).
2. It allocates a **bigger blob** on the **managed heap** where the actual `Order` object lives. The heap is shared by the whole application and is cleaned up by the **Garbage Collector (GC)**, not by you calling `free` like in C++.

So "reference type" really means: **the variable holds an address; the object lives elsewhere.**

Value types (`int` , `struct`) are different: small ones often live **inline** in the stack frame or **inside** another object, without a separate heap allocation.

Why Principal interviews care: Trading UIs create and destroy thousands of small objects per second (ticks, UI updates). Understanding heap vs stack explains GC pauses, allocation rates, and why SIG engineers obsess over structs and pooling in hot paths.

What every heap object contains

Object Header (sync block index, etc.)	8 bytes (64-bit)
Method Table Pointer (TypeHandle)	8 bytes — "what type am I?"
Field data (references + value fields)	variable size

The **Method Table** is the CLR's vtable: it tells the runtime how big the object is, which methods it has, and how to find virtual methods. When you call a virtual method, the CPU indirect-jumps through this table — slightly more expensive than a direct call.

The **object header** also supports **thin locking**: when you `lock(obj)` , the CLR first tries to store lock state in the header without extra allocations. Under contention, it escalates to a **SyncBlock** — an extra structure. That is why Principals say: **don't lock on random short-lived objects**; use a dedicated `private readonly object _gate = new();` .

Stack vs heap — practical table

Location	What lives there	Who cleans it up

Stack (per thread)	Local variables, return addresses, **references** to heap objects	Automatically when method returns
Managed heap	All `class` instances, boxed values, arrays, strings	Garbage Collector
Inside an object	`struct` fields stored **inline** in the parent object	When parent is collected

Mental model: Stack = fast, tiny, automatic cleanup per call. Heap = flexible, shared, GC-managed.

SIG / trading angle

- Every `new QuoteUpdate()` in a tight loop contributes to **Gen0 GC pressure**.
- Locking on tick DTOs created per message can create **SyncBlock churn**.
- Large `byte[]` buffers for market data may land on the **Large Object Heap (LOH)** — different GC rules (see Part II).

Interview phrases

> *"A local reference is on the stack; the object it points to is on the heap. I care about allocation rate because GC pauses can freeze a trader's UI during market open."*

---

2. class vs struct vs record vs record struct

Plain English

C# gives you four related tools. The choice is **not** style — it affects **memory, copying, equality, and GC**.

Keyword	Lives mainly on	Equality default	Typical mental model
`class`	Heap	Same instance = equal	"Entity with identity" — this order, that order
`struct`	Stack or inline	Bitwise copy	"Small packet of data" — a price level

`record` (class)	Heap	Same **values** = equal	"Immutable message / DTO"
`record struct`	Stack or inline	Same **values** = equal	"Small immutable value"

Identity vs value: Two `class` variables can point to the **same** order on the heap. Two `struct` variables are **independent copies** — changing one does not change the other.

class — reference type (expanded)

```
class Order
{
    public long Id;
    public string Symbol;
}

var a = new Order { Id = 1, Symbol = "AAPL" };
var b = a;
b.Id = 2;
// a.Id is also 2 — same object
```

When to use (Principal level):

- The thing has **identity** over time (order state machine: New Working Filled).
- You need **polymorphism** (`List<IOrderHandler>`).
- The payload is **large or variable-size** and copying would be expensive.
- You need **`null`** to mean "not present."

Cost: Every `new` hits the heap GC tracks it until unreachable.

struct — value type (expanded)

```
struct PriceLevel
{
    public decimal Price;
    public int Quantity;
}

var p1 = new PriceLevel { Price = 100.50m, Quantity = 500 };
```

```
var p2 = p1;
p2.Quantity = 0;
// p1.Quantity is still 500
```

Copy semantics: Assignment copies **all bytes** of the struct. A "fat" struct (many decimals, many fields) copied on every method call can cost more than a heap reference.

Rule of thumb: Hot-path structs often stay under ~16–32 bytes; **always profile** if unsure.

readonly struct: Compiler helps prevent accidental mutation after construction — important because mutating `list[i].Qty = 5` on a struct in a list is a classic bug (you mutate a copy).

ref struct: Special structs (`Span<T>`) that **cannot** go on the heap or cross `await` — stack-only, zero-copy views into memory.

record — reference type with value equality (expanded)

```
record Trade(long Id, decimal Price, string Symbol);

var t1 = new Trade(1, 100m, "AAPL");
var t2 = new Trade(1, 100m, "AAPL");
Console.WriteLine(t1 == t2); // true — same data
```

The compiler generates `Equals` , `GetHashCode` , `ToString` , and `with` for non-destructive updates:

```
var t3 = t1 with { Price = 101m }; // t1 unchanged
```

Still heap-allocated. Good for API boundaries and events; not automatically "free" in hot loops.

record struct — best of both for small immutables

```
readonly record struct BidAsk(decimal Bid, decimal Ask);
```

Value semantics + no identity + stack-friendly if small. Excellent for normalized quotes in an aggregator.

Decision matrix for SIG-style systems

Scenario	Choose	Why
Live `Order` entity with state transitions	`class`	Identity; mutable lifecycle
Tick snapshot in feed handler	`readonly record struct`	No alloc; value equality for tests
REST response body	`record`	Immutability + generated equality
Dictionary key (small, immutable)	`readonly record struct`	Good hash if fields are stable

Interview phrases

> *"I use class when identity and lifecycle matter; struct or record struct for high-frequency immutable snapshots; record for service-layer DTOs where value equality reduces bugs."*

---

3. Boxing, Unboxing, and ref struct

Plain English

Boxing wraps a value type (e.g. `int`) in a **heap object** so it can be treated as `object` or an interface. You pay: allocation + copy + eventual GC.

Unboxing copies the value back out and checks the type — wrong type throws `InvalidCastException`.

Why this still matters: Older APIs (`ArrayList`, non-generic collections, `SomeMethod(object o)`) box constantly. Modern generic code (`List<int>`) avoids most boxing — but **struct interface** still boxes.

Common boxing triggers

```
object o = 42;                  // boxes
IEnumerable<int> weird = 42;    // would box if valid
list.Add(42);                   // non-generic ArrayList
```

```
void Log(object value) { }
Log(42);                    // boxes at call site
```

Span and ref struct — the modern fix

`Span<T>` is a **window** into memory (array, stack buffer, native memory) **without** allocating a new object on the heap. Parsing market data with `ReadOnlySpan<byte>` avoids per-field strings.

Restriction: Cannot `await` with a `Span` local — the compiler enforces that the span must not outlive the stack frame across async suspension.

SIG angle

In a feed parser, `price.ToString()` and boxing in hot loops are silent killers. Prefer struct DTOs, spans, and symbol IDs as `int`.

---

4. String Internals & String Interning

Plain English

A C# `string` is **immutable**: once created, its characters never change. “Changing” a string always creates a **new** string on the heap.

Literal interning: When you write `"AAPL"` in source code, the compiler stores it once. At runtime, all references to that literal can share **the same object**:

```
string a = "AAPL";
string b = "AAPL";
ReferenceEquals(a, b); // often true for literals
```

Runtime-built strings are not interned by default:

```
string c = symbolFromFeed + monthCode;
ReferenceEquals(a, c); // false
```

string.Intern: Forces a runtime string into the intern pool. **Danger:** If you intern every unique order ID from a feed, the intern table grows forever and is never released

memory leak.

Better patterns for trading

- Parse symbol once store `int InstrumentId` in your catalog.
- Use `ReadOnlySpan<char>` for parsing without allocating substrings where possible.
- Use `StringBuilder` (or pools) for repeated formatting in warm paths — not `+` in a loop.

Part II — Garbage Collection

5. GC Fundamentals: Reachability & Roots

Plain English — the one sentence to remember

The GC collects objects that are no longer reachable from any "root."

Reachable = there exists a chain of references from a root to the object. Unreachable = eligible for collection (though collection may not happen immediately).

.NET does not use reference counting. Reference counting would fail on cycles (`A  B  A`). .NET uses **tracing**: start from roots, walk all references, mark everything you can reach; what's unmarked can be reclaimed.

What counts as a GC root?

Think of roots as **places the runtime knows to start searching**:

1. **Local variables and evaluation stack** on each thread (while a method is running).
2. **Static fields** — live for app lifetime, e.g. `static Dictionary<int, Instrument> Cache` .
3. **GC handles** — `GCHandle.Alloc` pins or prolongs life for interop.
4. **Thread-static** and special runtime structures.
5. **Finalizer queue** — objects waiting for `Finalize()` are temporarily roots (see §10).
6. **Interop** objects (COM, pinned buffers).

At a **GC safe point** (method prologue, loop back-edge), threads cooperate so the GC can scan stacks accurately.

Object graph — picture it

```
Roots    OrderCache    Order #123    LineItems    …
                                    Instrument "AAPL"

Orphan: TickDto #999 (no path from any root)    eligible for collection
```

Cycles

If `A` references `B` and `B` references `A`, but **nothing** from roots reaches `A`, both are collected. Tracing handles cycles; reference counting would not.

What keeps an object alive when you don't expect it?

- **Static event** still subscribed: `MarketDataHub.OnTick += vm.Handle` — the hub holds the delegate, the delegate holds `vm`, VM never dies classic WPF leak.
- **Timer** or **Closure** capturing `this` in a long-lived callback.
- **Cache** without eviction holding references forever.

Interview answer (expanded)

> *"Eligibility means no strong reference path from roots. The GC traces from thread stacks, statics, and handles, marks reachable objects, then reclaims the rest. It's not deterministic — I reduce allocations in hot paths so collections are rare and short, because stop-the-world pauses hurt trading UIs."*

---

6. GC Algorithms: Mark, Sweep, Compact

Plain English

Full garbage collection is roughly:

1. **Stop the world (mostly)** — pause app threads at safe points so stacks don't change mid-scan.
2. **Mark** — from every root, visit every reachable object and mark it "alive."
3. **Sweep** — dead objects become free memory (free lists).
4. **Compact (often for young gens)** — slide live objects together, update all pointers that referenced moved objects. This removes "Swiss cheese" holes in memory.

Why compact? After compacting, allocation is often a simple **bump pointer** ("next free slot") — very fast for short-lived objects.

Phase-by-phase (for interview whiteboard)

Phase	What happens
Suspend	Threads paused at safe points
Mark	Graph traversal from roots; cost ~ O(live objects)
Plan	Decide new addresses if compacting
Compact	Move objects; fix references
Sweep	Link dead regions into free lists
Resume	App continues

Young generations (Gen0/1) assume **most objects die young**, so collecting them often is cheap and frees lots of space.

---

7. Generational GC & Write Barriers

Plain English

Observation: Most objects die young (temporary strings, LINQ iterators, message wrappers). Old objects (singleton caches, instrument master) tend to stay.

So the heap is split into **generations**:

Gen	Nickname	Intuition
0	Nursery	Brand-new allocations; collected very often
1	Middle	Survived at least one Gen0 collection
2	Old	Long-lived; collected rarely and expensively

Promotion: Survive Gen0 promoted to Gen1 eventually Gen2.

The cross-generation reference problem

Suppose a **Gen2** singleton cache holds a reference to a **Gen0** tick object. When we collect Gen0 only, we might miss that tick if we don't also scan Gen2.

Write barrier: Whenever code stores a reference into an old object (e.g. `cache.Latest = newTick`), the JIT inserts a small bookkeeping hook — often marking a **card** in a **card table** — meaning "something in this memory region may point to young objects." Gen0 collection scans dirty cards instead of the entire Gen2 heap.

Cost: Tiny overhead on reference writes to old objects; huge win vs scanning all old memory every Gen0.

---

8. LOH, POH, and Segments

Plain English

Large Object Heap (LOH): Objects **≥ ~85,000 bytes** go here (large arrays, big buffers). Historically LOH was collected with Gen2 and **not compacted often fragmentation** over long runs.

.NET 5+: LOH compaction can be enabled; **DATAS** helps the runtime adapt generation sizes.

Pinned Object Heap (POH): Long-lived pinned buffers can go here so pinning doesn't fragment the main heap.

Pinning: `fixed` or `GCHandleType.Pinned` tells GC **don't move this object** while pinned. Too much pinning blocks compaction.

SIG practice: Rent large buffers from `ArrayPool<byte>.Shared` and return them; don't allocate `new byte[200_000]` per message.

---

9. GC Modes: Workstation vs Server, Background GC

Plain English

Workstation GC: One heap, tuned for client apps — often lower latency on few cores.

Server GC: One heap **per logical CPU** — higher throughput under parallel allocation (typical on trader desktops with many cores and background feed threads).

Background GC: Lets the app run **during parts of Gen2 collection**, reducing long pauses. Still has stop-the-world moments, but shorter than full blocking Gen2.

Latency modes (`GCSettings.LatencyMode`): e.g. `LowLatency` temporarily tries to avoid full GC — use sparingly; memory pressure can force a bigger collection later.

Don't call `GC.Collect()` in production to "fix" UI — fix allocation rate instead.

---

10. Finalization, Weak References, and GC Handles

Finalization — why `IDisposable` exists

If a class has a destructor (`~MyClass()`), the GC **doesn't reclaim it immediately** when unreachable. It enqueues it for **finalization** — one more GC cycle, non-deterministic thread runs `Finalize()` .

Problems: Delayed cleanup, extra GC pressure, undefined order.

Pattern:

```
public void Dispose()
{
   Dispose(true);
   GC.SuppressFinalize(this); // "don't finalize me — I cleaned up"
}
```

Native handles (sockets, file handles) need deterministic `Dispose` , not "maybe later."

WeakReference

Holds a reference that **does not keep the object alive**. Useful for caches that shouldn't prevent collection, and WPF weak event patterns.

GCHandle types (summary)

Type	Effect
Normal	Strong — prevents collection
Pinned	Strong + fixed address for native interop

Weak	Doesn't prevent collection

---

11. GC Tuning & Diagnostics

What to measure (and what good looks like)

- **% Time in GC** — low single digits for healthy UI apps under load.
- **Gen0 collections/sec** — high = allocation pressure; investigate hot loops.
- **Gen2 frequency** — should be rare in steady state; frequent Gen2 = leaks or LOH churn.
- **Pause times** — correlate with "UI froze for 200ms" reports.

Tools

- **dotMemory** — who allocates, dominator tree.
- **PerfView** — GC events, pause markers.
- **dotnet-counters** — live `gc-heap-size`, `gen-0-gc-count`.
- **dotnet-gcdump** — snapshot of what's on the heap.

Mitigation checklist (actionable)

1. Profile before optimizing — prove where allocations happen.
2. Replace class DTOs with structs in hot paths where appropriate.
3. `ArrayPool`, object pools for messages.
4. Fix event/timer leaks (Part V).
5. Choose Server vs Workstation GC deliberately for deployment.

Part III — Concurrency & Threading

12. Threads, ThreadPool, and Scheduling

Plain English

A **thread** is an OS-level "line of execution" with its own stack (~1 MB reserved). Creating many threads is expensive; switching between them costs CPU (context switch).

The **ThreadPool** is a shared pool of worker threads. When you `Task.Run(...)`, work usually queues here. The pool grows and shrinks based on load (**hill climbing**).

Thread pool starvation: If all pool threads block waiting on something (e.g. `.Result` on async code), new work sits in the queue — app feels hung. **Fix:** async all the way; don't block pool threads.

Dedicated threads still make sense for: STA COM apartments, a dedicated blocking feed reader, or pinning to a CPU core for latency (advanced).

CPU affinity: Restrict process to specific cores so hot threads and their memory stay on the same NUMA node — relevant on server-class hardware.

---

13. Task, async/await, and State Machines

Plain English

`async` / `await` does **not** mean "run on another thread." It means: **"I can pause here without blocking the thread, and resume when work completes."**

When you `await ReadFromSocketAsync()`:

- If the read isn't done, the method **returns** an incomplete `Task` and registers a **continuation**.
- The thread is free to do other work (UI message pump, other requests).
- When the socket read completes, the continuation runs — often on a thread pool thread, or back on the **UI thread** if a `SynchronizationContext` was captured.

Compiler magic (conceptual)

`async Task<int> M()` becomes a **state machine** with:

- An integer state ("which `await` am I past?")
- Fields for locals that must survive across `await`
- A `MoveNext()` method the runtime calls to resume

ConfigureAwait — the rule everyone forgets

- **UI code / ViewModels** updating bindings: usually **need** to resume on UI thread default `ConfigureAwait(true)` or don't use false.
- **Library code** with no UI: `ConfigureAwait(false)` avoids capturing UI context unnecessarily.

Common foot-gun

```
// BAD on UI thread — blocks message pump
var data = GetDataAsync().Result;
```

Use `await GetDataAsync()` from an async event handler instead.

---

14. ValueTask and IAsyncDisposable

ValueTask avoids allocating a `Task` object when the operation **often completes synchronously** (e.g. cache hit). Rules: **consume once**; don't await the same `ValueTask` twice unless converted to `Task`.

IAsyncDisposable — `await using` for async cleanup of streams and connections.

---

15. Synchronization Primitives

Plain English guide

Tool	When to use
`lock`	Simple mutual exclusion — only one thread in critical section
`ReaderWriterLockSlim`	Many readers OR one writer (reference data cache)
`SemaphoreSlim`	Limit concurrency (max 5 parallel downloads)
`Interlocked`	Atomic increment/compare-exchange — counters, lock-free
`Mutex`	Cross-process single instance

Never lock on `string` literals, `typeof(T)`, or public `this` — collision and deadlock risk.

---

16. .NET Memory Model & Lock-Free Programming

Plain English

CPUs and compilers **reorder** memory operations for speed. Without synchronization, thread B might not see thread A's writes when you expect.

.NET fixes this with:

- `lock` — release/acquire semantics
- `Interlocked` operations
- `Volatile.Read` / `Volatile.Write`

Publish snapshot pattern (very common at SIG):

Writer builds new immutable state off-thread, then publishes one reference:

```
Volatile.Write(ref _snapshot, newSnapshot);
```

Readers:

```
var snap = Volatile.Read(ref _snapshot);
```

Readers never take locks; they see old or new snapshot, not a half-updated structure.

---

17. Concurrent Collections

ConcurrentDictionary — thread-safe dictionary with lock striping. Good general purpose; not always fastest for **read-heavy** if a plain dictionary + periodic immutable snapshot is enough.

Immutable collections — every "update" creates new version; readers keep old version safely.

ConcurrentQueue — common for producer/consumer between feed thread and workers.

---

18. Deadlock

Plain English

Deadlock = two threads each hold a lock the other needs, forever.

Prevention: Global lock order — always acquire Account, then Order, then Position. Document it. Use `Monitor.TryEnter` with timeout in infrastructure code.

Detection: Visual Studio Parallel Stacks, `dotnet-dump`, PerfView blocked time.

SIG preference: Message passing and single-writer per partition beats nested locks on hot paths.

Part IV — Performance & Allocations

19. Hidden Allocations & JIT Optimizations

Plain English — start here if you're rusty

In C#, you might think "I didn't call `new`, so I didn't allocate." That is often **wrong**. Many language features allocate **hidden objects** on the heap. The Garbage Collector then has more work to do, which can cause **micro-stutters** or occasional **long pauses** — unacceptable when a trader is watching a live book.

Common hidden allocators:

What you write	What actually happens on the heap
`list.Where(x => x > limit)`	Iterator object + often a **closure** holding `limit`
`async Task M() { await X(); }`	State machine (sometimes heap-promoted), `Task`, continuations
`s += piece` in a loop	New string every iteration (strings are immutable)
`foreach` over non-optimized IEnumerable	Enumerator object
`params object[]` or boxing a `int` into `object`	Boxed value on heap
`$"Price {p}"`	Interpolated string builder (sometimes stack-optimized in .NET 6+)

The JIT (Just-In-Time compiler) turns IL bytecode into native machine code at runtime. Modern .NET uses **tiered compilation**:

1. **Tier 0** — compile quickly with few optimizations so code runs fast enough to start.
2. After a method is called many times, **Tier 1** recompiles with inlining, better register use, etc.
3. **Dynamic PGO** (.NET 7+) can further optimize hot branches.

What JIT can fix: Remove redundant bounds checks, inline small methods, devirtualize sealed calls.

What JIT cannot fix: A `new List<T>()` you wrote in source, LINQ chains, or boxing you introduced. **Allocation rate is an architecture and coding-style problem**, not something the JIT magically removes.

How to investigate: dotMemory, PerfView allocation ticks, `dotnet-counters` for GC heap size. **Always measure Release, Tier-1 warmed up** — Debug builds and cold JIT mislead you.

SIG / trading angle

A feed handler parsing 200k messages/sec with LINQ and string splits will allocate gigabytes per minute Gen0 collections every few milliseconds correlated UI hitches if anything shares the process.

Interview phrases

> *"I'd profile allocation rate per message first. Hidden costs are usually closures, async state machines, and string work — not the obvious `new` calls."*

---

20. Span, Memory, ArrayPool, and stackalloc

Plain English

Problem: Arrays and strings on the heap trigger GC. For hot parsing paths, you want to work on **memory you already have** without copying.

Span<T> is a **window** into contiguous memory — an array slice, a stack buffer, or native memory. It is a **ref struct** (stack-only, cannot be stored in a field of a normal class, cannot cross `await`). You pass slices without allocating sub-arrays.

```
ReadOnlySpan<byte> payload = buffer.AsSpan(offset, length);
int priceTicks = BinaryPrimitives.ReadInt32LittleEndian(payload.Slice(8, 4));
```

Memory<T> is the heap-friendly cousin — use when you need to hold a buffer reference across `await` (e.g. `ReadAsync(Memory<byte>)`).

ArrayPool<T>.Shared rents arrays larger than requested. **Critical rules:**

- Always `Return` in `finally` — leaks pool slots if you don't.
- Never use the array after `Return` — another thread may rent it.
- Don't assume exact length — you get `>=` requested size.

stackalloc allocates on the **thread stack** (fast, no GC). Fine for small scratch buffers (256 bytes, 1 KB). The whole thread stack is ~1 MB — don't `stackalloc` megabytes.

SIG / trading angle

Fixed-layout binary ticks: `ReadOnlySpan&lt;byte&gt;` + `BinaryPrimitives` = parse without `BitConverter`, without substring, without split.

Interview phrases

> *"Span for synchronous parse on rented ArrayPool buffers; Memory for async I/O. Return pooled arrays in finally."*

---

21. Processing 1 Million Messages/sec

Plain English

"One million messages per second" sounds like a micro-optimization challenge. At Principal level it is a **system design** question: throughput = how fast you allocate, lock, copy, and branch — not raw CPU GHz.

Think in stages:

```
Network  Parse  Normalize  Shard writer  (optional) UI snapshot
```

Each arrow is a **queue** or **ring buffer** with a **policy** when full.

Design checklist (explain each aloud):

1. **Binary fixed layout** — know byte offsets for price, qty, sequence. No XML/JSON in the hot path unless you accept the cost.
2. **Object pooling** — reuse message structs/shells; Gen0 dies young but collection still has cost.
3. **Partition by symbol** — `symbolId % N` N single-writer threads. Same symbol always same shard order preserved without global lock.
4. **Backpressure** — display quotes: `DropOldest` when queue full. Orders: `Wait` or spill to disk — never silently drop.
5. **Batch** — apply 64 updates, then wake UI once; amortize cache lines and locks.
6. **Measure** — messages/sec, queue depth P99, **Gen0 collections/sec**, cache misses (PerfView/hardware counters).

Clarifying question for interview: "Is P99 latency 50 µs or 50 ms?" Display path coalesces to 60 Hz; risk path may not.

SIG / trading angle

SIG-style desktop apps often combine **write-heavy feed** with **read-heavy WPF**. The million/sec path must **never** block on Dispatcher or allocate per tick for the grid.

Interview phrases

> *"I'd partition single-writer, pool message shells, and separate compliance tape from coalesced display state. I'd measure queue depth and GC before tuning micro-ops."*

Part V — WPF

22. WPF Architecture & Threading (Dispatcher)

Plain English — if you haven't done desktop UI in years

WPF (Windows Presentation Foundation) is Microsoft's desktop UI framework for rich trading workstations. Under the hood it still behaves like classic Windows UI: one **UI thread** owns all controls.

That thread runs a **message loop** (message pump): dequeue message handle click run layout paint repeat. If you do heavy work **on the UI thread**, the loop stops processing messages window **freezes** ("Not responding").

Golden rule: Only the UI thread touches `Button`, `TextBox`, `DataGrid`, or any `DependencyObject`. Background threads must **marshal** work back via the **Dispatcher**.

API	Behavior	Risk
`Dispatcher.Invoke(action)`	Run on UI thread **now**; caller **blocks** until done	Deadlock if UI thread waits on caller
`Dispatcher.BeginInvoke(action)`	Queue work; caller continues	Safer for feed UI updates
`await` in code-behind/VM	After await, often resumes on UI thread via `SynchronizationContext`	Library code should use `ConfigureAwait(false)`

STA (Single-Threaded Apartment): WPF's `Main` must be `[STAThread]` because COM (clipboard, some dialogs, older integrations) requires it. This is not optional trivia — wrong apartment breaks clipboard paste in production.

Render thread: WPF may use a separate thread for composition (milcore). You still must **create and mutate** controls on the UI thread.

SIG / trading angle

Feed threads must **never** set `textBox.Text = ...` directly. Pattern: update in-memory model on feed thread `BeginInvoke` or coalesced timer on UI thread diff visible cells.

Interview phrases

> *"UI thread owns DependencyObjects. I marshal with BeginInvoke or coalesced snapshots — Invoke only when I truly need synchronous completion, and I avoid cross-thread waits that deadlock."*

---

23. Logical Tree, Visual Tree, and Rendering Pipeline

Plain English

WPF maintains two trees:

- **Logical tree** — your XAML structure (`Window` `Grid` `ListBox` `ListBoxItem`).

- **Visual tree** — what actually gets drawn after **templates** expand. A `Button` might become `Border` + `ContentPresenter` + `TextBlock`.

`VisualTreeHelper.GetChild` walks the visual tree — useful in debugging (Snoop) and hit-testing.

When something changes size or content, layout runs:

1. **Measure pass** — parent asks child: "Given this much space, how big do you want to be?" Children answer with `DesiredSize`. Bottom-up.
2. **Arrange pass** — parent assigns final rectangles. Top-down.
3. **Render** — visuals draw. Hardware path uses DirectX when **RenderCapability.Tier >= 2**.

InvalidateMeasure / InvalidateArrange / InvalidateVisual mark subtrees dirty. A price column updating `Width` every tick forces layout for many elements — **avoid**. Prefer fixed row height, update `Text` or brush only, or animate `RenderTransform` (GPU-friendly).

SIG / trading angle

Options chain with variable row heights defeats virtualization benefits. Fixed row height + virtualizing panel is standard for large grids.

Interview phrases

> *"Measure then arrange then render. High-frequency price updates should not change layout-affecting properties every tick."*

---

24. DependencyProperty & Property System Internals

Plain English

A normal property is a field on your object. WPF's **DependencyProperty** system stores values in a **centralized, sparse table** per `DependencyObject` so the framework can:

- **Bind** XAML `{Binding Price}` to a ViewModel property
- Apply **styles** and **triggers** without your code running
- **Animate** values over time
- **Inherit** values down the tree (e.g. `FontSize` on parent flows to children when marked `Inherits=true`)

Effective value precedence (highest wins):

1. Local value (code or XAML)
2. Triggers (including data triggers)
3. Styles
4. Theme
5. Default from `FrameworkPropertyMetadata`

When a DP changes, **metadata callbacks** can fire (`OnPriceChanged` `InvalidateMeasure`). Data binding installs a **BindingExpression** as the effective value source — it subscribes to `INotifyPropertyChanged` and pushes updates into the DP.

Why this matters for performance: 5,000 cells × 4 bound columns × 1,000 updates/sec = millions of property system operations. Coalesce at the model layer.

SIG / trading angle

Prefer one `INotifyPropertyChanged` on a row VM with batched refresh over per-cell binding storms during market open.

Interview phrases

> *"DPs enable binding and styling; each change can cascade layout. I batch source notifications instead of firing PropertyChanged per tick per cell."*

---

25. Data Binding Internals

Plain English

Data binding connects a **source** (ViewModel `Price`) to a **target** (TextBlock `Text` DependencyProperty).

At runtime WPF:

1. Resolves the path (`Price` , `Order.Symbol` , etc.) — reflection first, then cached accessors.
2. Creates a **BindingExpression** on the target DP.
3. Subscribes to source change notifications (`INotifyPropertyChanged` , `CollectionChanged`).

Update modes:

Mode	Meaning
OneWay	Source UI (typical display)
TwoWay	UI edits write back (forms)
OneTime	Set once at load
OneWayToSource	UI source only

ObservableCollection<T> raises `CollectionChanged` on add/remove/replace. The grid's view refreshes — expensive if you mutate the collection constantly. For live quotes, **mutate properties on existing row objects** or replace snapshot reference — don't clear/add thousands of rows per second.

INotifyPropertyChanged: Each `PropertyChanged` event can trigger target update. Batch: raise one event after updating many fields, or use a "refresh token" property.

SIG / trading angle

Blotter rows are stable identity (order id); update status/qty in place. Quote grids may rebuild visible slice from snapshot each frame — not full `ObservableCollection` reset.

Interview phrases

> *"BindingExpression listens to INPC. I avoid collection churn; I update existing row state or publish immutable snapshots for the visible window."*

---

26. Virtualization & Large Data Sets (500k Rows)

Plain English

UI virtualization means the `ListBox` / `DataGrid` only **instantiates containers for visible rows** (~viewport + small cache), recycling them as you scroll. Scrolling through 500k rows does not create 500k `ListBoxItem` objects.

Requirements (common gotchas):

- `VirtualizingPanel.IsVirtualizing="True"`
- `ScrollViewer.CanContentScroll="True"` (item scrolling, not pixel scrolling)
- **Stable item height** strongly preferred — variable height forces remeasure and kills performance

Recycling mode reuses the same container; only updates `Content` — less allocation than standard mode.

Data virtualization is separate: even with UI virtualization, if your `ItemsSource` is `List<Order>` with 500k **loaded objects**, memory explodes. Implement `IList` with `Count = 500000` but **fetch page from DB** on indexer access; evict pages far from scroll position.

Custom drawing: For charts or order-book depth, subclass and `OnRender` — draw geometry directly without 10,000 `Rectangle` elements.

SIG / trading angle

Historical order search may need data virtualization; live chain uses UI virtualization + in-memory snapshot of visible symbols only.

Interview phrases

> *"UI virtualization limits visual tree size; data virtualization limits heap. I need both for 500k rows from SQL."*

---

27. High-Frequency UI Updates (1000 ticks/sec)

Plain English

A market feed can deliver **many updates per symbol per second**. Human eyes and monitors cannot usefully display 1000 refreshes/sec (60 Hz monitor ≈ 16 ms per frame).

Pattern:

```
Every tick (1000/sec)     update in-memory LatestQuote[id]  (writer thread)
Display loop (60 Hz)      copy snapshot for visible ids      diff cells      bind once
```

Use `DispatcherTimer`, `CompositionTarget.Rendering`, or Rx `Sample(16ms)` — not per-tick `BeginInvoke`.

Two paths:

Path	Rate	Purpose
Risk / compliance	Every tick	Limits, logging, alerts

Display	30–60 Hz	Trader sees prices

Conflating them causes either wrong risk (if you coalesce risk) or frozen UI (if you bind every tick).

SIG / trading angle

Token-bucket or "latest wins" queue for display; separate durable append for compliance tape.

Interview phrases

> *"Model truth can be every tick; view state is coalesced to frame rate. I never bind the grid directly to raw observables firing 1000/sec."*

---

28. WPF Performance, Leaks, and Diagnostics

Plain English

WPF apps **leak memory** when long-lived objects hold references to short-lived ViewModels — the GC cannot collect the VM, and everything it references stays alive.

Classic leak sources:

Leak	Mechanism
`SomeStaticCache.Updated += vm.OnUpdate`	Static event holds delegate holds VM
`DispatcherTimer` not stopped	Timer holds callback holds VM
Binding to closed window	Target or source chain keeps graph alive
`Task` fire-and-forget capturing `this`	VM captured in closure

Fixes: Weak event pattern, `WeakReferenceMessenger`, unsubscribe in `Closed` / `Dispose`, clear bindings (`BindingOperations.ClearBinding`), stop timers.

Diagnostics:

- **Snoop** — inspect visual tree, binding errors, DP values live

- **dotMemory** — dominator tree: who references this ViewModel?
- **PresentationTraceSources** — trace binding failures to Output window
- **WPA/ETW** — UI thread CPU if layout/render dominates

SIG / trading angle

Traders leave workstations open for days. A 50 MB/hour leak becomes a 2 AM outage. Principal reviews focus on event lifetime and static caches.

Interview phrases

> *"I treat VM lifetime explicitly: unsubscribe static events, stop DispatcherTimer, clear bindings on close. dotMemory dominators for proof."*

Part VI — System Design

29. Real-Time Market Data Application

Plain English — the whole pipeline

Market data is a **firehose**: UDP multicast or TCP streams deliver binary messages thousands or millions of times per second. The desktop app must **parse**, **normalize**, **store latest state**, and **show** it without freezing.

Components (explain like you're talking to a rusty engineer)

1. Feed Handler

- Joins network feed (multicast group or TCP).
- Reads bytes into buffers (ideally pooled).
- Parses message format (fixed offsets — not CSV in hot path).
- Tracks **sequence numbers** — if gap detected, request resync or mark data **stale**.

2. Normalizer

- Converts venue-specific format your internal `Quote` / `Trade` model.
- Maps `"ESH5"` internal `InstrumentId = 42`.
- Applies entitlements (user allowed to see this instrument?).

3. Market Data Cache

- **Latest state only** for display: best bid, best ask, last trade, volume.
- Usually **one writer thread per partition** (e.g. by symbol hash) — no locks on every tick.

4. Publisher

- UI and other modules subscribe here — **not** to raw socket.
- Slow consumer? Coalesce or drop with metrics — **never** block feed thread indefinitely.

5. UI (WPF)

- Pulls snapshots at 30–60 Hz.
- Virtualized grids; diff updates, not full rebind per tick.

Failure modes (always mention in interview)

- Stale data after disconnect show banner, disable trading if policy requires live data.
- Gap in sequence snapshot + incremental recovery.
- Slow UI backpressure on publisher, not unbounded queue growth.

Technical depth — threading diagram

```
[Network thread]  parse  enqueue

[Writer shard 0..N-1]  update QuoteState (no UI, no locks across shards)

[Publisher]  coalesce / topic fan-out

[UI timer 60Hz]  snapshot visible ids  ViewModel diff  virtualized grid
```

Never let the UI thread join multicast or parse XML per packet.

Interview phrases

> *"I'd draw feed normalize partitioned cache publisher coalesced UI. Slow consumer gets coalesced or dropped with metrics, never blocks the socket thread."*

---

30. Order Management System (OMS)

Plain English

An OMS tracks **orders from idea to exchange to fill**. Traders care about **correctness** more than microsecond UI polish.

Typical flow:

```
Trader clicks Buy
    Client validation (qty, tick size)
    Pre-trade risk (limits, credit, position)
    Persist to journal (WAL) BEFORE sending to exchange
    Send to exchange via gateway
    Receive ack / reject / fills
    Update blotter state
```

Write-Ahead Log (WAL): Append “Order X submitted” to durable disk **before** you tell the trader “sent.” If process crashes, replay journal on restart.

Idempotency: `ClientOrderId` UUID — duplicate submit rejected.

UI rule: Don’t show “Filled” optimistically — show “Pending” until exchange confirms.

Partitioning: Scale by account, symbol, or desk — each partition single-writer.

Technical depth — order state machine

```
Created  PendingRisk  Accepted  Working  PartialFill  Filled
                Rejected
    Working  PendingCancel  Cancelled
```

Every transition is **audited** (who, when, why). Pre-trade risk may be sync (block until pass) or async (pending state) — **business policy**, not purely technical.

Smart order routing awareness: splits, iceberg, pegged — Principal knows names even if not implementing SOR.

Interview phrases

> *"Journal before send, idempotent client order id, pessimistic UI until exchange ack. I'd partition writers and never show filled without confirmation."*

---

31. Low-Latency Trading GUI — Latency Budgets

Plain English

A **latency budget** assigns a time allowance to each step so you can measure where slowness lives:

Step	Display path (typical)
Socket app	venue-dependent
Parse message	microseconds
Update cache	microseconds
Wait for UI frame	up to 16 ms (60 Hz)
Bind + render	few ms

Execution path (click Buy) may have stricter, separate budgets than **display path** (show flickering prices).

Instrument with timestamps at each boundary; report P50 and P99, not just average.

Worked example (options chain)

2000 strikes × 4 columns, 500 aggregate updates/sec:

Stage	Budget	Notes
NIC process	venue-specific	Hardware timestamp if available
Parse	~5 µs/msg	Fixed binary
Shard insert	~2 µs	Queue depth metric
UI coalesce	~16 ms	By design at 60 Hz
Diff + render	few ms	Virtualized grid

Perceived tick-to-screen ~16–25 ms — acceptable for **display**; **Buy** button may use a separate low-latency path with stricter budget.

Correlate **GC pause P99** with UI hitch metrics — Principals prove causation, not guess.

Interview phrases

> *"I split display vs execution budgets. I instrument each hop and report P99, including GC correlation."*

Part VII — Coding & Leadership

32. LRU Cache — Full Implementation & Variants

Plain English

Imagine a desk with only **10 slots** for instrument detail panels. When you open an 11th, you close the panel you **haven't looked at longest** — that is **LRU (Least Recently Used)**.

Computers implement this with:

- **Dictionary** — "given key, find the node instantly" (O(1) average).
- **Doubly linked list** — "move this key to front on access" and "remove tail when evicting" (O(1)).

Every `TryGet` **touches** the item (moves to front). Every `Put` when full **evicts** the tail.

When LRU is wrong: If your working set is **all 50,000 listed symbols** and you must show any of them live, LRU evicts useful data constantly — use a full in-memory table sized to the universe, not LRU.

```
public sealed class LruCache<TKey, TValue> where TKey : notnull
{
    private readonly int _capacity;
    private readonly Dictionary<TKey, LinkedListNode<(TKey Key, TValue Value)>> _map;
    private readonly LinkedList<(TKey Key, TValue Value)> _list;

    public LruCache(int capacity)
```

```
    {
        if (capacity <= 0) throw new ArgumentOutOfRangeException(nameof(capacity));
        _capacity = capacity;
        _map = new Dictionary<TKey, LinkedListNode<(TKey, TValue)>>(capacity);
        _list = new LinkedList<(TKey, TValue)>();
    }

    public bool TryGet(TKey key, out TValue value)
    {
        if (!_map.TryGetValue(key, out var node))
        {
            value = default!;
            return false;
        }
        _list.Remove(node);
        _list.AddFirst(node);
        value = node.Value.Value;
        return true;
    }

    public void Put(TKey key, TValue value)
    {
        if (_map.TryGetValue(key, out var existing))
        {
            _list.Remove(existing);
            existing.Value = (key, value);
            _list.AddFirst(existing);
            return;
        }
        if (_map.Count >= _capacity)
        {
            var lru = _list.Last!;
            _list.RemoveLast();
            _map.Remove(lru.Value.Key);
        }
        var node = new LinkedListNode<(TKey, TValue)>((key, value));
        _list.AddFirst(node);
        _map[key] = node;
    }
}
```

Thread-safe variant: Wrap in one `lock` for interviews; mention per-bucket locks only if asked about extreme concurrency.

Interview phrases

> *"Hash map plus doubly linked list gives O(1) get and put. I'd only use LRU when working set exceeds capacity — not when the universe fits in memory."*

---

33. Order Book & BBO

Plain English

The **order book** is all resting buy (**bid**) and sell (**ask**) orders at price levels. **BBO (Best Bid and Offer)** is the **best bid** (highest price buyers pay) and **best ask** (lowest price sellers accept). **Spread** = best ask − best bid.

Feeds send **deltas**: "at price 100.50, bid quantity is now 500" or "remove level 100.25."

SortedDictionary approach: Bids sorted descending, asks ascending. Update O(log n). Simple, correct — fine for interview and many products.

Tick-indexed array: If prices are on a fixed tick grid (e.g. pennies from $50 to $150), map price index. Update O(1), maintain `_bestBidTick` pointer when levels go empty. Costs memory O(tick range).

Principal trade-off: ES futures might use tick array; illiquid options with wide strike spacing might use tree/dictionary.

Interview phrases

> *"I'd clarify tick grid and update rate. SortedDictionary for clarity; tick array when range is bounded and hot path demands O(1)."*

---

34. Token Bucket & Rate Limiting

Plain English

A **token bucket** is a jar of tokens refilling at steady rate **R** (e.g. 10 per second), max capacity **B**. Each API call consumes one token. No token **reject** or **wait**.

Unlike “fixed window” counters, it allows **smooth bursts** up to capacity while limiting average rate.

UI coalescing variant: Don’t reject ticks — **overwrite** latest quote; a 60 Hz timer “consumes” one display token and paints. Semantically rate-limits **screen work**, not **market truth**.

```
public sealed class TokenBucket
{
    private readonly double _capacity;
    private readonly double _refillPerSecond;
    private double _tokens;
    private long _lastRefillTicks;

    public TokenBucket(double capacity, double refillPerSecond)
    {
        _capacity = capacity;
        _refillPerSecond = refillPerSecond;
        _tokens = capacity;
        _lastRefillTicks = Environment.TickCount64;
    }

    private void Refill()
    {
        var now = Environment.TickCount64;
        var elapsed = (now - _lastRefillTicks) / 1000.0;
        if (elapsed <= 0) return;
        _tokens = Math.Min(_capacity, _tokens + elapsed * _refillPerSecond);
        _lastRefillTicks = now;
    }

    public bool TryConsume(double tokens = 1)
    {
        Refill();
        if (_tokens < tokens) return false;
        _tokens -= tokens;
        return true;
    }
}
```

Interview phrases

> *"Token bucket shapes average rate with controlled burst. For WPF I'd rate-limit display commits, not the feed ingestion path."*

---

35. Behavioral — Unpopular Decisions & Mentoring

Plain English — what Principals actually evaluate

Technical depth gets you in the room; **judgment and leadership** get you the offer. They want stories where you:

- Made a **correct but unpopular** call (removed a feature, delayed launch, chose boring tech).
- **Influenced without authority** — convinced trading desk, compliance, or another team.
- **Mentored** seniors and juniors — not by doing their design for them.

STAR template (walk through slowly)

Situation: "Our options chain froze at market open. P99 UI latency hit 2 seconds; desk escalated."

Task: "As tech lead, I owned diagnosis and a fix that wouldn't slip the regulatory release."

Action: "I profiled GC and bindings — 40% time in Gen0 from per-tick string formatting. I proposed coalescing display to 60 Hz; desk initially resisted 'less fresh data.' I showed risk path still saw every tick; only grid coalesced. Piloted on one desk for a week."

Result: "P99 UI latency dropped to 80 ms; zero trading incidents from stale risk; two other desks adopted. Release stayed on date."

Mentoring philosophy

Ask questions that build judgment:

- "What breaks at **10×** message rate?"
- "Who gets paged at **2 a.m.** if this fails?"
- "What's the **rollback** in one sentence?"
- "What would you **measure** to know you're done?"

Interview phrases

> *"I optimize for operability and measured outcomes, not popularity. I mentor by stress-testing assumptions, not by writing their design."*

Part VIII — Extended Deep Dives (Descriptive Summaries)

Sections 36–64 in the comprehensive reference go deeper on JIT, GC walkthroughs, WPF routed events, full market-data/OMS documents, Rx.NET, and testing. Below is the **rusty-reader translation** of the highest-yield topics.

36–37. GC Allocation Path & Collection Walkthrough

Allocation: `new` usually bumps a pointer in a Gen0 segment — extremely fast until the segment fills, then Gen0 GC runs. Large objects go to LOH with different compaction rules.

Walkthrough intuition: GC starts from **roots**, marks reachable objects, compacts young generations, updates pointers. An object only in a cycle with no root path is collected — unlike reference counting.

38. IDisposable vs Finalizer

Dispose = deterministic cleanup **now** (files, sockets, native handles). **Finalizer** = "run later on GC thread if programmer forgot" — slow, unordered, keeps object alive an extra GC cycle. Always `GC.SuppressFinalize(this)` when you dispose properly.

39–40. JIT & Assembly Loading

Code starts as IL; RyuJIT compiles to native in tiers. **AssemblyLoadContext** (.NET Core+) lets you load plugins in isolated contexts — unload only if no leaked references. Trading platforms use this for reference-data plugins with careful lifetime discipline.

41–42. ThreadPool Starvation & async Allocations

Blocking `.Result` on thread pool threads while queue needs workers global stall. Async reduces blocking but **incomplete awaits** allocate Tasks and continuations — `ValueTask` helps hot paths that usually complete sync.

43–46. Memory Barriers, Lock-Free Queues, RW Lock

Publish **immutable snapshots** with `Volatile.Write` for lock-free readers. **MPSC queues** between feed parsers and aggregators; **Disruptor** pattern (ring buffer + sequence numbers) for extreme throughput — busy-spin vs block is a **business** latency decision.

47–51. WPF Internals

Routed events bubble/tunnel through the tree. **VirtualizingStackPanel** recycles containers via `ItemContainerGenerator`. **Data virtualization** uses custom `IList` to page from SQL.

52–58. Coding Deep Dives

Full **order book**, **token bucket**, **merge K sorted level lists**, **sequence gap detection** — see Part VII and comprehensive doc for code. **Sequence gaps** trigger snapshot recovery on market data feeds.

59–64. Rx.NET, MVVM Toolkit, Testing, Traps

Rx `Sample(16ms)` per symbol for UI. **WeakReferenceMessenger** avoids static leaks. Test ViewModels with xUnit; replay **golden feed files** for parser regression. **Trap:** binding every tick; **trap:** optimistic order filled UI.

Part IX — Read-Heavy vs Write-Heavy Systems & Caching

65. Read-Heavy Systems: Architecture & Patterns

Plain English

A system is **read-heavy** when it mostly **answers questions** and rarely **changes data**. Think: thousands of traders reading instrument names, tick sizes, and entitlements; only an admin team writes when a new listing appears.

Ratios like **100:1** or **10,000:1** read-to-write are common for reference data and dashboards.

The mistake to avoid: Serving every grid cell refresh with a SQL `JOIN` across five tables. Instead, **optimize the read path** even if the write path stays normalized in the database.

Core pattern — separate write path from read path:

```
Writers (few)    Authoritative database (source of truth)
                 async projection / cache warm / replica
Readers (many)   Fast copy (memory snapshot, Redis, read replica)
```

Immutable snapshot (in-process): When a catalog changes, build a **new** dictionary and swap one `volatile` reference. Readers never lock; they see old or new, never half-updated. Ideal for **instrument master** — wrong for **every-tick quotes** (too many writes to copy trees).

Other read-heavy tools: Read replicas (SQL follower), materialized views (pre-joined blotter rows), CQRS read models, denormalized DTOs (`OrderBlotterRow` with symbol name already on it).

Failure modes: Stale cache (show version + TTL), **cache stampede** (many threads miss at once — fix with **single-flight**), unbounded cache growth (cap + LRU).

Interview phrases

> *"Read-heavy means I never join at UI refresh time. I project on write or publish immutable snapshots for reference data."*

---

66. Write-Heavy Systems: Architecture & Patterns

Plain English

Write-heavy means the system **constantly mutates state**: every market tick, fill, audit log line, or metric sample.

The enemy is **many threads writing the same structure** — locks pile up, deadlocks appear, latency spikes.

Core pattern — partition + single writer:

```
Feed  hash(symbol) % N  Queue[i]  ONE writer thread[i]  Shard state[i]
```

Same symbol always hits same shard **order preserved** without a global lock on every message.

Bounded queues between stages force you to decide policy when overloaded:

Policy	Use when
Block	Orders — cannot drop
Drop oldest	Display quotes — latest matters
Coalesce	Overwrite pending update for same key

Append-only log (WAL): For orders and audit, **append to disk before ack**. Crash recovery = replay log. Ticks may optionally batch to time-series storage.

Allocation discipline: Struct messages, `ArrayPool`, no LINQ in parse loop — GC is a write-heavy failure mode.

Interview phrases

> *"Write-heavy means single-writer per partition, bounded queues with explicit drop/block policy, and WAL before ack for anything traders trust."*

---

67. Caching Fundamentals: Concepts & Metrics

Plain English

A **cache** is a faster, smaller copy of data **closer to the consumer** than the **origin** (SQL server, REST API, file share).

Hit — key found, data returned from cache (fast).

Miss — not found or expired load from origin (slow) usually store in cache for next time.

Eviction — remove entry because cache is full (capacity) or TTL expired.

Key metrics:

```
Hit Rate = Hits / (Hits + Misses)
Origin Load = Miss Rate × Read QPS
```

Example: 100,000 reads/sec, 98% hit rate origin sees ~2,000/sec.

Working set — keys actually accessed in a time window. If working set > cache size, you churn entries and hit rate collapses. Trading data is often **Zipfian** — few hot symbols dominate.

Negative caching: Cache "instrument not found" for 30 seconds so repeated bad lookups don't hammer SQL.

Interview phrases

> *"I size cache to working set, measure hit rate and origin QPS, and negative-cache missing keys."*

---

68. Cache Replacement Policies (LRU, LFU, FIFO, TTL, ARC)

Plain English

When the cache is **full** and you need a slot, **which entry do you throw away?**

Policy	Intuition	Good for	Weakness
FIFO	Oldest inserted leaves	Simple buffers	Ignores popularity
LRU	Longest since last use leaves	UI navigation, recent symbols	Full table scan evicts hot set
LFU	Lowest use count leaves	Stable hot keys (SPY, QQQ)	Old junk never dies without aging
TTL	Time expires entry	Config, entitlements	Stale until clock runs out

Random	Random victim	Cheap approximate	Unpredictable
ARC	Adapts recency vs frequency	Mixed workloads	Complex to implement

ARC / W-TinyLFU — mention when interviewer says "LRU fails on scan." Shows Principal breadth; .NET `IMemoryCache` is LRU-ish, not ARC.

Interview phrases

> *"LRU for temporal locality in UI caches; LFU with aging for always-hot underlyings; TTL when freshness has a known bound."*

---

69. Cache-Aside, Read-Through, Write-Through, Write-Back, Write-Around

Plain English — the five patterns everyone confuses

Cache-aside (lazy loading) — **you** (application code) manage cache:

- Read: check cache miss read DB put cache return.
- Write: write DB first **invalidate or update** cache.

Most common in C# services. **Bug:** forgetting invalidation on write.

Read-through — cache library loads on miss; app always calls `cache.Get`. Loader lives in cache layer.

Write-through — write updates **cache and DB together** before returning. Strong consistency; slower writes.

Write-back (write-behind) — write cache only, ack fast, flush DB later. High throughput; **data loss risk** if node dies — never for orders without separate WAL.

Write-around — write DB only, skip cache (or invalidate). Good for bulk EOD imports that won't be read immediately.

Pattern	Orders?	Quote display?	Instrument lookup?

Cache-aside	Invalidate on state change	Coalesced in-memory	Yes
Write-through	If must read own writes instantly	Rare	Sometimes
Write-back	**No** (without WAL)	Metrics only	Risky

Interview phrases

> *"Default cache-aside with explicit invalidation on write. Orders use WAL + durable write-through to journal — not write-back."*

---

70. Cache Invalidation Strategies

Plain English

Phil Karlton: "There are only two hard things... cache invalidation and naming things."

TTL — entry dies after N minutes. Simple; data can be wrong until expiry. Add **jitter** so keys don't all expire at once (thundering herd).

Event-driven — on `InstrumentUpdated`, remove key from local cache and publish to Redis pub/sub. Minimizes staleness after admin edits.

Version / ETag — store `(value, version)`; reject cache if source version newer.

Write-through — no invalidation needed for that key if every write updates cache.

Full flush — nuclear option on schema migration; spikes origin load.

Single-flight (stampede fix): On miss, only **one** thread loads origin; others `await` same `Task`. Critical at startup and TTL expiry waves.

Interview phrases

> *"TTL with jitter for entitlements; event invalidation for instrument master; single-flight on miss to protect SQL."*

---

71. In-Process Caching in C# (Implementations)

Plain English

IMemoryCache (`Microsoft.Extensions.Caching.Memory`) — DI-friendly, supports absolute + sliding expiration, `SizeLimit` with per-entry size, and **CancellationToken** to bulk-invalidate groups.

ConcurrentDictionary + TTL wrapper — full control; you implement eviction policy yourself.

HybridCache (.NET 8+) — L1 memory + L2 distributed (Redis) with one API.

ConditionalWeakTable — derived data tied to object lifetime; when `Instrument` is collected, cached formatting goes too — good for memory-sensitive derived strings.

Single-flight pattern — wrap `GetOrCreateAsync` so concurrent misses don't each hit SQL.

Principal depth: register **post-eviction callbacks** for metrics; set **Size** on entries when using `SizeLimit` .

Interview phrases

> *"IMemoryCache for services with size limits and cancellation tokens; immutable snapshot for hot in-process catalogs; single-flight on loader."*

---

72. Multi-Level & Distributed Caching

Plain English

Caches stack in layers:

```
L1: IMemoryCache in desktop app     (~microseconds)
L2: Redis on LAN                    (~sub-ms)
L3: SQL / origin                    (~milliseconds+)
```

On read: try L1 L2 origin; populate on the way back.

Redis uses: key-value with TTL, hashes for quote fields, pub/sub for invalidation.
Serialization cost (JSON vs protobuf) matters at scale.

Desktop SIG app: L1 in ViewModel/service; optional L2 for shared reference data API; SQL origin for admin truth.

Interview phrases

> *"Multi-level with explicit populate on miss; Redis pub/sub for cross-process invalidation after admin writes."*

---

73. CQRS, Materialized Views & Read Models

Plain English

CQRS splits **commands** (writes that change state) from **queries** (reads that never mutate).

Write model stays normalized and safe. **Events** (`OrderFilled`) project into a **read model** optimized for grids: `BlotterRow` with everything the UI needs in one row object.

Cost: Eventual consistency — read model may lag milliseconds behind write. Need reconciliation jobs if projection bugs.

Worth it at SIG: Complex blotter + search. **Overkill:** simple admin CRUD. **Not classic CQRS:** in-memory latest quote per symbol (single-writer cache).

Interview phrases

> *"CQRS when read shape differs from write shape — blotter rows from order events. Market data latest state is single-writer memory, not event-sourced CQRS."*

---

74. Trading & WPF Examples: Read vs Write Paths

Plain English — four concrete stories

A. Instrument master (read-heavy): SQL authoritative; app holds immutable snapshot; refresh on `CatalogUpdated` event; WPF binds once per refresh — not per lookup.

B. Live quotes (write ingest, read display): Write path: feed queue writer updates `QuoteState[]` . Read path: 60 Hz timer copies visible symbols diff cells. Compliance tape = separate append-only write path.

C. Blotter (mixed): Order events project to read model; grid reads in-memory rows; writes moderate via OMS events.

D. EOD PnL (read-heavy batch): Overnight job write-around bulk load; morning thousands of analysts read; Redis L2 + long TTL until next EOD.

Interview phrases

> *"Same app mixes write-heavy feed with read-heavy UI — I never use one pattern everywhere; I classify each path."*

---

75. Choosing a Strategy: Decision Framework

Plain English — five steps aloud in interview

1. **Classify** read QPS vs write QPS, latency SLO, consistency, durability.
2. **Write architecture:** partition + single-writer + queue policy; WAL if traders trust it.
3. **Read architecture:** snapshot / replica / materialized view / cache-aside.
4. **Cache policy:** LRU vs TTL vs full universe in memory; single-flight on miss.
5. **Operability:** metrics (hit rate, queue depth, GC, replication lag), alerts, runbooks.

Closing template (memorize)

> *"Feed ingress is write-heavy — partition by symbol, single-writer, bounded queues with drop-oldest for display. WPF is read-heavy — coalesce to frame rate, virtualize, diff cells. Reference data — immutable snapshot with event invalidation. SQL lookups — cache-aside with single-flight. Orders — WAL before ack; never write-back without durability."*

Appendix A — GC Quick Reference (Plain English)

Question	Answer in one breath
When is object collected?	No strong reference path from any GC root
How does GC find live objects?	Trace from roots; mark reachable; sweep/compact rest

Why generations?	Most objects die young — scan young heap often, old rarely
What is write barrier?	When old object points to young, remember that card for partial GC
LOH?	Big arrays (~85KB+) — different compaction; use pools
Finalizer?	Last-resort cleanup — prefer `IDisposable`

Appendix B — Principal Interview Phrases

- “Let me clarify latency tolerance and whether we can drop or coalesce before I design.”
- “Display path coalesces; risk path may consume every tick.”
- “I’d measure Gen0 rate and GC pause P99 before micro-optimizing.”
- “Single-writer per partition plus immutable snapshot for readers.”
- “Cache-aside with invalidation on write; single-flight on miss.”

Appendix C — Master Checklist

Use after reading expanded sections; check off when you can **explain aloud** without notes.

- ☐ Heap vs stack vs reference vs value
- ☐ Object header, MethodTable, SyncBlock
- ☐ class / struct / record decision matrix
- ☐ Boxing triggers and string intern risks
- ☐ Why GC traces from roots (not reference counting)
- ☐ Mark-sweep-compact and generational hypothesis
- ☐ Write barriers and LOH / pinning
- ☐ Workstation vs Server GC; Background GC
- ☐ IDisposable vs finalizer
- ☐ async state machine; ConfigureAwait on UI vs library
- ☐ lock ordering; immutable snapshot publish
- ☐ Hidden allocations; Span / ArrayPool

- [ ] Dispatcher; measure/arrange/render
- [ ] DP precedence; binding / INPC storms
- [ ] UI vs data virtualization; tick coalescing
- [ ] Market data pipeline end-to-end
- [ ] OMS WAL, idempotency, state machine
- [ ] LRU, order book, token bucket (can code on whiteboard)
- [ ] Read-heavy vs write-heavy classification
- [ ] Cache-aside, write-through, write-back trade-offs
- [ ] LRU / LFU / TTL / ARC when asked
- [ ] Single-flight and event invalidation
- [ ] STAR behavioral story with metrics

Expanded study guide (this file): `C:\Users\Admin\sig-principal-csharp-wpf-prep-expanded.md`

Dense technical reference with full §36–64 code: `C:\Users\Admin\optiver-principal-csharp-wpf-prep-comprehensive.md`